

Dancing Links

本页面将介绍精确覆盖问题、重复覆盖问题，解决这两个问题的算法「X 算法」，以及用来优化 X 算法的双向十字链表 Dancing Link。本页也将介绍如何在建模的配合下使用 DLX 解决一些搜索题。

精确覆盖问题

定义

精确覆盖问题（英文：Exact Cover Problem）是指给定许多集合 $S_1, S_2, \dots, S_n$ 以及一个集合 X ，求满足以下条件的无序多元组：

1. X 是 S_i 的并集。
2. X 中每个元素恰好属于 S_i 中的一个。
3. X 中元素个数等于 X 中集合个数。

解释

例如，若给出

则 $\{S_1, S_2, S_3\}$ 为一组合法解。

问题转化

将 S_i 中的所有数离散化，可以得到这么一个模型：

给定一个 01 矩阵，你可以选择一些行（row），使得最终每列（column）¹都恰好有一个 1。举个例子，我们对上文中的例子进行建模，可以得到这么一个矩阵：

其中第 i 行表示着 S_i ，而这一行的每个数依次表示 $x_1, x_2, \dots, x_n$ 。

实现

暴力 1

一种方法是枚举选择哪些行，最后检查这个方案是否合法。

因为每一行都有选或者不选两种状态，所以枚举行的时间复杂度是 $O(2^n)$ 的；

而每次检查都需要 $O(n)$ 的时间复杂度。所以总的复杂度是 $O(2^n \cdot n)$ 。

► 实现

暴力 2

考虑到 01 矩阵的特殊性质，每一行都可以看做一个 n 位二进制数。

因此原问题转化为

给定 n 个 m 位二进制数，要求选择一些数，使得任意两个数的与都为 0，且所有数的或为 $(2^m - 1)$ 。 tmp 表示的是截至目前被选中的二进制数的或。

因为每一行都有选或者不选两种状态，所以枚举的时间复杂度为 $O(2^n)$ ；

而每次计算 tmp 都需要 $O(m)$ 的时间复杂度。所以总的复杂度为 $O(2^n \cdot m)$ 。

► 实现

重复覆盖问题

重复覆盖问题与精确覆盖问题类似，但没有对元素相似性的限制。下文介绍的 [X 算法](#) 原本针对精确覆盖问题，但经过一些修改和优化（已标注在其中）同样可以高效地解决重复覆盖问题。

X 算法

Donald E. Knuth 提出了 X 算法 (Algorithm X)，其思想与刚才的暴力差不多，但是方便优化。

过程

继续以上文中提到的例子为载体，得到一个这样的 01 矩阵：

1. 此时第一行有 4 个 1，第二行有 3 个 1，第三行有 2 个 1，第四行有 1 个 1，第五行有 1 个 1，第六行有 1 个 1。选择第一行，将它删除，并将所有 1 所在的列打上标记；
2. 选择所有被标记的列，将它们删除，并将这些列中含 1 的行打上标记（重复覆盖问题无需打标记）；
3. 选择所有被标记的行，将它们删除；

这表示这一行已被选择，且这一行的所有 1 所在的列不能有其他 1 了。

于是得到一个新的小 01 矩阵：

4. 此时第一行（原来的第二行）有 3 个 1，第二行（原来的第四行）有 2 个 1，第三行（原来的第五行）有 1 个 1。选择第一行（原来的第二行），将它删除，并将所有 1 所在的列打上标记；
5. 选择所有被标记的列，将它们删除，并将这些列中含 1 的行打上标记；
6. 选择所有被标记的行，将它们删除；

这样就得到了一个空矩阵。但是上次删除的行 1 0 1 1 不是全 1 的，说明选择有误；

- 回溯到步骤 4，考虑选择第二行（原来的第四行），将它删除，并将所有 所在的列打上标记；
- 选择所有被标记的列，将它们删除，并将这些列中含 的行打上标记；
- 选择所有被标记的行，将它们删除；

于是我们得到了这样的一个矩阵：

- 此时第一行（原来的第五行）有 个，将它们全部删除，得到一个空矩阵；
- 上一次删除的时候，删除的是全 的行，因此成功，算法结束。

答案即为被删除的三行：。

强烈建议自己模拟一遍矩阵删除、还原与回溯的过程后，再接着阅读下文。

通过上述步骤，可将 X 算法的流程概括如下：

- 对于现在的矩阵，选择并标记一行，将 添加至 中；
- 如果尝试了所有的 却无解，则算法结束，输出无解；
- 标记与 相关的行 和 （相关的行和列与 [X 算法](#) 中第 2 步定义相同，下同）；
- 删除所有标记的行和列，得到新矩阵；
- 如果 为空，且 为全，则算法结束，输出被删除的行组成的集合；

如果 为空，且 不全为，则恢复与 相关的行 以及列，跳转至步骤 1；

如果 不为空，则跳转至步骤 1。

不难看出，X 算法需要大量的「删除行」、「删除列」和「恢复行」、「恢复列」的操作。

一个朴素的想法是，使用一个二维数组存放矩阵，再用四个数组分别存放每一行与之相邻的行编号，每次删除和恢复仅需更新四个数组中的元素。但由于一般问题的矩阵中 0 的数量远多于 1 的数量，这样做的空间复杂度难以接受。

Donald E. Knuth 想到了用双向十字链表来维护这些操作。

而在双向十字链表上不断跳跃的过程被形象地比喻成「跳跃」，因此被用来优化 X 算法的双向十字链表也被称为「Dancing Links」。

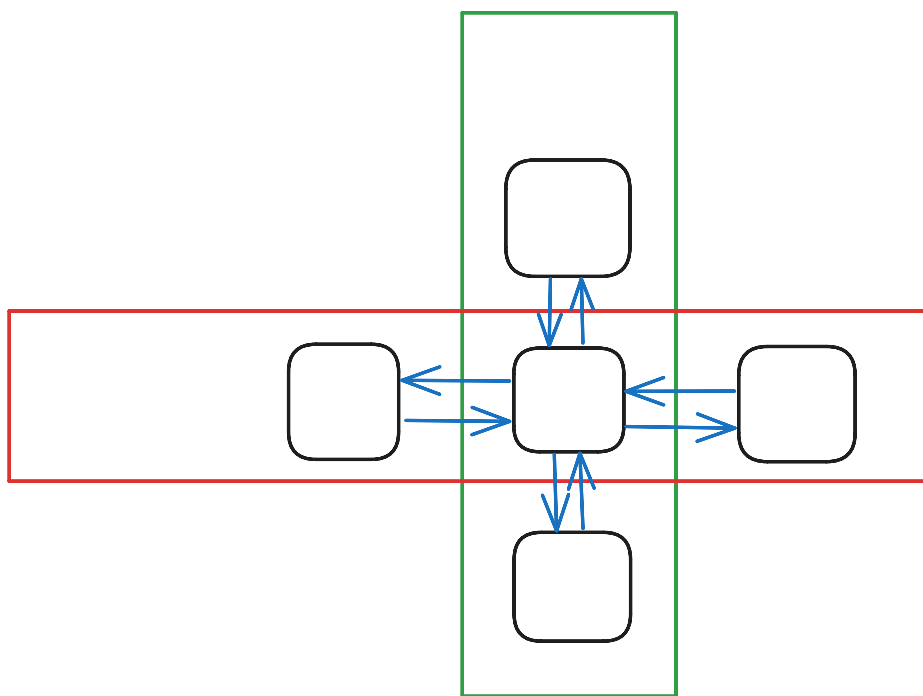
Dancing Links 优化的 X 算法

预编译命令

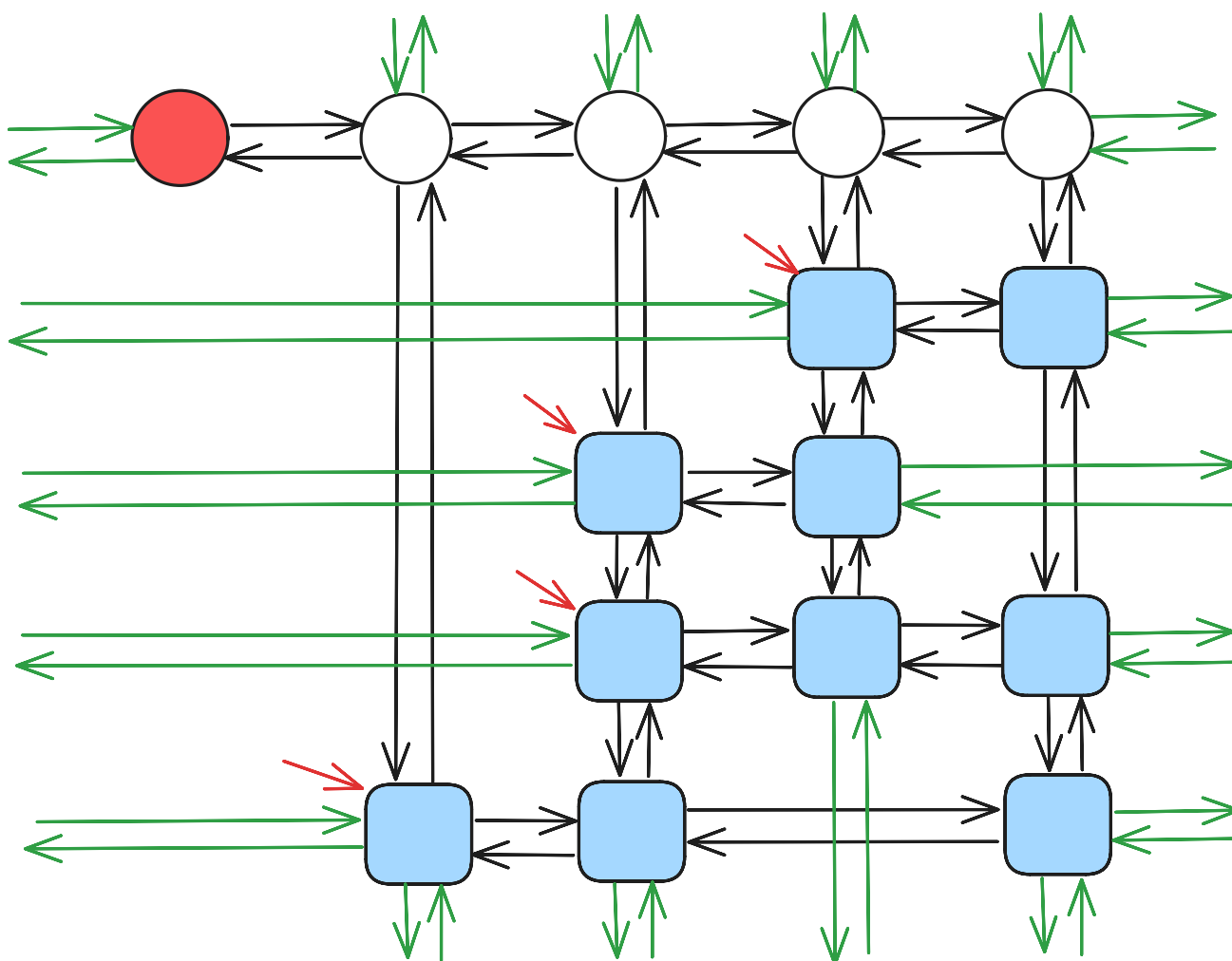
```
1 #define IT(i, A, x) for (i = A[x]; i != x; i = A[i])
```

定义

双向十字链表中存在四个指针域，分别指向上、下、左、右的元素；且每个元素 在整个双向十字链表系中都对应着一个格子，因此还要表示 所在的列和所在的行，如图所示：



大型的双向链表则更为复杂：



每一行都有一个行首指示，每一列都有一个列指示。

行首指示为 `first[]`，列指示是我们新建的 个哨兵结点。值得注意的是，行首指示并非是链表中的哨兵结点。它是虚

拟的，类似于邻接表中的 `first[]` 数组，**直接指向** 这一行中的首元素。

同时，每一列都有一个 `siz[]` 表示这一列的元素个数。

特殊地，号结点无右结点等价于这个 Dancing Links 为空。

```
1 constexpr int MS = 1e5 + 5;
2 int n, m, idx, first[MS], siz[MS];
3 int L[MS], R[MS], U[MS], D[MS];
4 int col[MS], row[MS];
```

过程

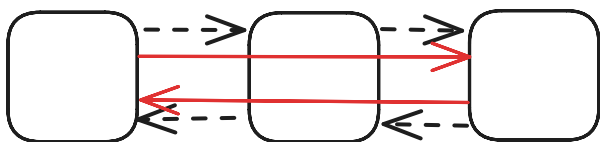
remove 操作

`remove(c)` 表示在 Dancing Links 中删除第 列以及与其相关的行和列。

先将 删除，此时：

- 左侧的结点的右结点应为 的右结点。
- 右侧的结点的左结点应为 的左结点。

即 $L[R[c]] = L[c]$, $R[L[c]] = R[c]$ 。



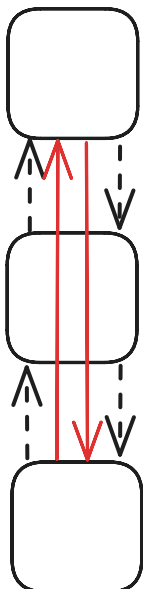
然后顺着这一列往下走，把走过的每一行都删掉。

如何删掉每一行呢？枚举当前行的指针，此时：

- 上方的结点的下结点应为 的下结点。
- 下方的结点的上结点应为 的上结点。

注意要修改每一列的元素个数。

即 $U[D[j]] = U[j]$, $D[U[j]] = D[j]$, $--siz[col[j]]$ 。



remove 函数的代码实现如下：

▼ 实现

```
1 void remove(const int &c) {
2     int i, j;
3     L[R[c]] = L[c], R[L[c]] = R[c];
4     // 顺着这一列从上往下遍历
5     IT(i, D, c)
6     // 顺着这一行从左往右遍历
7     IT(j, R, i)
8     U[D[j]] = U[j], D[U[j]] = D[j], --siz[col[j]];
9 }
```

recover 操作

recover(c) 表示在 Dancing Links 中还原第 列以及与其相关的行和列。

recover(c) 即 remove(c) 的逆操作，这里不再赘述。

值得注意的是，recover(c) 的所有操作的顺序与 remove(c) 的操作恰好相反。

recover(c) 的代码实现如下：

▼ 实现

```
1 void recover(const int &c) {
2     int i, j;
3     IT(i, U, c) IT(j, L, i) U[D[j]] = D[U[j]] = j, ++siz[col[j]];
4     L[R[c]] = R[L[c]] = c;
5 }
```

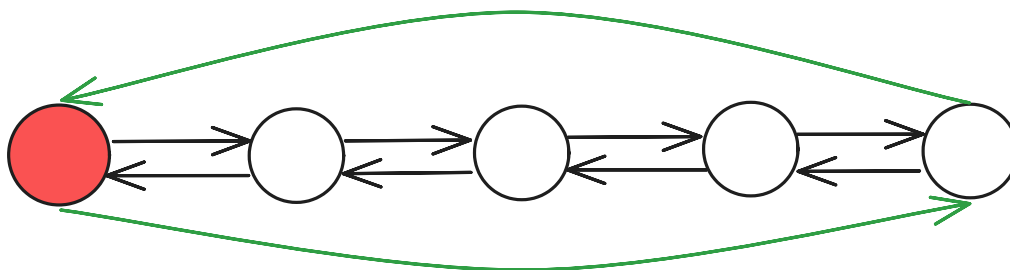
build 操作

build(r, c) 表示新建一个大小为 r ，即有 r 行， c 列的 Dancing Links。

新建 c 个结点作为列指示。

第 i 个点的左结点为 $L[i]$ ，右结点为 $R[i]$ ，上结点为 $U[i]$ ，下结点为 $D[i]$ 。特殊地， $L[0]$ 结点的左结点为 $L[0]$ ， $R[0]$ 结点的右结点为 $R[0]$ 。

于是我们得到了一个环状双向链表：



这样就初始化了一个 Dancing Links。

build(r, c) 的代码实现如下：

▼ 实现

```

1 void build(const int &r, const int &c) {
2     n = r, m = c;
3     for (int i = 0; i <= c; ++i) {
4         L[i] = i - 1, R[i] = i + 1;
5         U[i] = D[i] = i;
6     }
7     L[0] = c, R[c] = 0, idx = c;
8     memset(first, 0, sizeof(first));
9     memset(siz, 0, sizeof(siz));
10 }

```

insert 操作

`insert(r, c)` 表示在第 `r` 行，第 `c` 列插入一个结点。

插入操作分为两种情况：

- 如果第 `r` 行没有元素，那么直接插入一个元素，并使 `first[r]` 指向这个元素。

这可以通过 `first[r] = L[idx] = R[idx] = idx;` 来实现。

- 如果第 `r` 行有元素，那么将这个新元素用一种特殊的方式与 `L[idx]` 和 `R[idx]` 连接起来。

设这个新元素为 `idx`，然后：

- 把 `idx` 插入到 `L[idx]` 的正下方，此时：

- 下方的结点为原来 `L[idx]` 的下结点；
- 下方的结点（即原来 `L[idx]` 的下结点）的上结点为 `idx`；
- `idx` 的上结点为 `R[idx]`；
- `idx` 的下结点为 `D[idx]`。

注意记录 `idx` 的所在列和所在行，以及更新这一列的元素个数。

```

1 col[++idx] = c, row[idx] = r, ++siz[c];
2 U[idx] = c, D[idx] = D[c], U[D[c]] = idx, D[c] = idx;

```

强烈建议读者完全掌握这几步的顺序后再继续阅读本文。

- 把 `idx` 插入到 `R[idx]` 的正右方，此时：

- 右侧的结点为原来 `R[idx]` 的右结点；
- 原来 `R[idx]` 的结点的左结点为 `idx`；
- `idx` 的左结点为 `L[idx]`；
- `idx` 的右结点为 `R[idx]`。

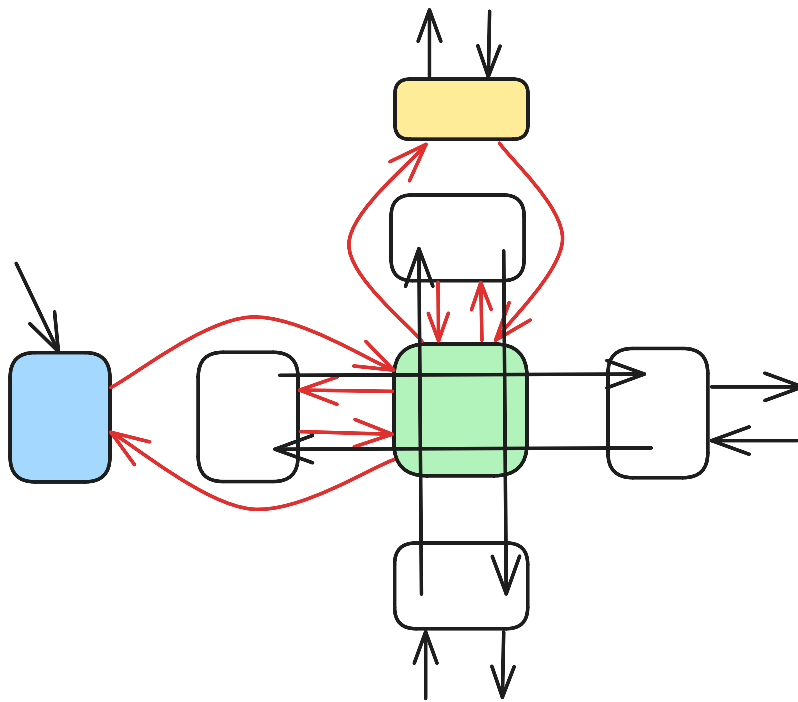
```

1 L[idx] = first[r], R[idx] = R[first[r]];
2 L[R[first[r]]] = idx, R[first[r]] = idx;

```

强烈建议读者完全掌握这几步的顺序后再继续阅读本文。

`insert(r, c)` 这个操作可以通过图片来辅助理解：



留心曲线箭头的方向。

`insert(r, c)` 的代码实现如下：

▼ 实现

```

1 void insert(const int &r, const int &c) {
2     row[++idx] = r, col[idx] = c, ++siz[c];
3     U[idx] = c, D[idx] = D[c], U[D[c]] = idx, D[c] = idx;
4     if (!first[r])
5         first[r] = L[idx] = R[idx] = idx;
6     else {
7         L[idx] = first[r], R[idx] = R[first[r]];
8         L[R[first[r]]] = idx, R[first[r]] = idx;
9     }
10 }

```

dance 操作

`dance()` 即为递归地删除以及还原各个行列的过程。

1. 如果 号结点没有右结点，那么矩阵为空，记录答案并返回；
2. 选择列元素个数最少的一列，并删掉这一列；
3. 遍历这一列所有有 的行，枚举它是否被选择；
4. 递归调用 `dance()`，如果可行，则返回；如果不可行，则恢复被选择的行；
5. 如果无解，则返回。

`dance()` 的代码实现如下：

▼ 实现


```

1 bool dance(int dep) {
2     int i, j, c = R[0];
3     if (!R[0]) {
4         ans = dep;
5         return true;
6     }
7     IT(i, R, 0) if (siz[i] < siz[c]) c = i;
8     remove(c);
9     IT(i, D, c) {
10         stk[dep] = row[i];
11         IT(j, R, i) remove(col[j]);
12         if (dance(dep + 1)) return true;
13         IT(j, L, i) recover(col[j]);
14     }
15     recover(c);
16     return false;
17 }

```

其中 `stk[]` 用来记录答案。

注意我们每次优先选择列元素个数最少的一列进行删除，这样能保证程序具有一定的启发性，使搜索树分支最少。

对于重复覆盖问题，在搜索时可以用估价函数（与 [A*](#) 中类似）进行剪枝：若当前最好情况下所选行数超过目前最优解，则可以直接返回。

模板

► [模板代码](#)

性质

DLX 递归及回溯的次数与矩阵中的个数有关，与矩阵的等参数无关。因此，它的时间复杂度是指数级的，理论复杂度大概在左右，其中为某个非常接近于的常数，为矩阵中的个数。

但实际情况下 DLX 表现良好，一般能解决大部分的问题。

建模

DLX 的难点，不全在于链表的建立，而在于建模。

请确保已经完全掌握 DLX 模板后再继续阅读本文。

我们每拿到一个题，应该考虑行和列所表示的意义：

- 行表示决策，因为每行对应着一个集合，也就对应着选/不选；
- 列表示状态，因为第列对应着某个条件。

对于某一行而言，由于不同的列的值不尽相同，我们由不同的状态，定义了一个决策。

例题 1 [P1784 数独](#)

- 解题思路
- 参考代码

例题 2 [靶形数独](#)

- 解题思路

► 参考代码

例题 3 [「NOI2005」智慧珠游戏](#)

► 解题思路

► 参考代码

习题

- [SUDOKU - Sudoku](#)
- [「kuangbin 带你飞」专题三 Dancing Links](#)

外部链接

- [夜深人静写算法（九） - Dancing Links X（跳舞链）_WhereIsHeroFrom 的博客》](#)
- [跳跃的舞者，舞蹈链（Dancing Links）算法——求解精确覆盖问题 - 万仓一黍](#)
- [DLX 算法一览 - zhangjianjunab](#)
- [搜索：DLX 算法 - 静听风吟。](#)
- [《算法竞赛入门经典 - 训练指南》](#)

注释

-
1. （两岸用语差异）台湾：直行（column）、横列（row） [↩](#)
-

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [383494](#), [leverimmy](#), [CCXXXI](#), [Chrogeek](#), [EarthMessenger](#), [Enter-tainer](#), [Great-designer](#), [iamtwz](#), [lr1d](#), [kenlig](#), [Konano](#), [ksyx](#), [L1nkzz](#), [LeverImmy](#), [NachtgeistW](#), [opsiff](#), [ouuan](#), [SamZhangQingChuan](#), [StudyingFather](#), [Tiphereth-A](#), [W-RJ](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用